
Table of Contents

Drone Simulation & Paper Figure Replication	1
1. Physical Parameters and Simulation Settings	1
2. Exact Analytical Solution	2
3. Numerical Solvers: Euler, RK4, and ode45	3
4. Visualizations: Trajectory and Velocities	3
5. Numerical Error Analysis	5
6. Stability Analysis (Euler Step Sizes)	6
7. Parameter Sensitivity Analysis	7
8. Obstacle Avoidance Simulation	9
9. Flight Path Animation	9
10. Replicate Figure 1: Altitude Step Response	11
11. Replicate Figure 2: Attitude Tilt Angles	12
12. Replicate Figure 3: Single Obstacle Avoidance Trajectory	13
13. Replicate Figure 4: Multiple Obstacle Navigation Trajectory	15
=====	16

Drone Simulation & Paper Figure Replication

All project codes consolidated for simulation, analysis, and publishing.

```
clear;
clc;
close all;
```

1. Physical Parameters and Simulation Settings

Parameter used in the simulation

```
% Drone mass (kg)
params.mass = 1.5;

% Total thrust (N)
params.thrust = 20;

% Linear drag coefficient (N.s/m)
params.drag = 0.5;

% Gravity (m/s^2)
params.gravity = 9.81;

% Drone orientation
% Pitch angle (rad)
params.pitch = deg2rad(10);

% Roll angle (rad)
params.roll = deg2rad(5);

% Simulation parameters
```

```
% Start time (s)
params.t0 = 0;

% End time (s)
params.tf = 45;

% Default step size
params.h = 0.1;

% Initial conditions [x y z vx vy vz]
params.Y0 = [0;0;0;0;0;0];

disp('Parameters loaded successfully:');
disp(params);

Parameters loaded successfully:
    mass: 1.5000
   thrust: 20
    drag: 0.5000
 gravity: 9.8100
   pitch: 0.1745
    roll: 0.0873
     t0: 0
     tf: 45
      h: 0.1000
     Y0: [6x1 double]
```

2. Exact Analytical Solution

Calculates the analytical solution for the drone motion.

```
% Time vector
tAna = params.t0 : params.h : params.tf;
N = length(tAna);
YAna = zeros(6, N);

m = params.mass;
T = params.thrust;
k = params.drag;
g = params.gravity;
theta = params.pitch;
phi = params.roll;

% Constant accelerations
Ax = (T/m) * sin(theta) * cos(phi);
Ay = -(T/m) * sin(phi);
Az = (T/m) * cos(theta) * cos(phi) - g;

x0 = params.Y0(1); y0 = params.Y0(2); z0 = params.Y0(3);
vx0 = params.Y0(4); vy0 = params.Y0(5); vz0 = params.Y0(6);

for i = 1:N
    ti = tAna(i);
```

```
vx = (vx0 - Ax*m/k)*exp(-k*ti/m) + Ax*m/k;
vy = (vy0 - Ay*m/k)*exp(-k*ti/m) + Ay*m/k;
vz = (vz0 - Az*m/k)*exp(-k*ti/m) + Az*m/k;

x = x0 + (vx0 - Ax*m/k)*(m/k)*(1-exp(-k*ti/m)) + (Ax*m/k)*ti;
y = y0 + (vy0 - Ay*m/k)*(m/k)*(1-exp(-k*ti/m)) + (Ay*m/k)*ti;
z = z0 + (vz0 - Az*m/k)*(m/k)*(1-exp(-k*ti/m)) + (Az*m/k)*ti;

YAna(:,i) = [x; y; z; vx; vy; vz];
end

disp('Analytical solution computed.');
```

Analytical solution computed.

3. Numerical Solvers: Euler, RK4, and ode45

Solves the trajectory using all three numerical integration methods.

```
[tEuler, YEuler] = euler_solver(@drone_trajectory_dynamics, params);
[tRK4, YRK4]     = rk4_solver(@drone_trajectory_dynamics, params);
[tODE45, YODE45] = ode45_solver(@drone_trajectory_dynamics, params);

disp('Euler solver completed successfully.');
```

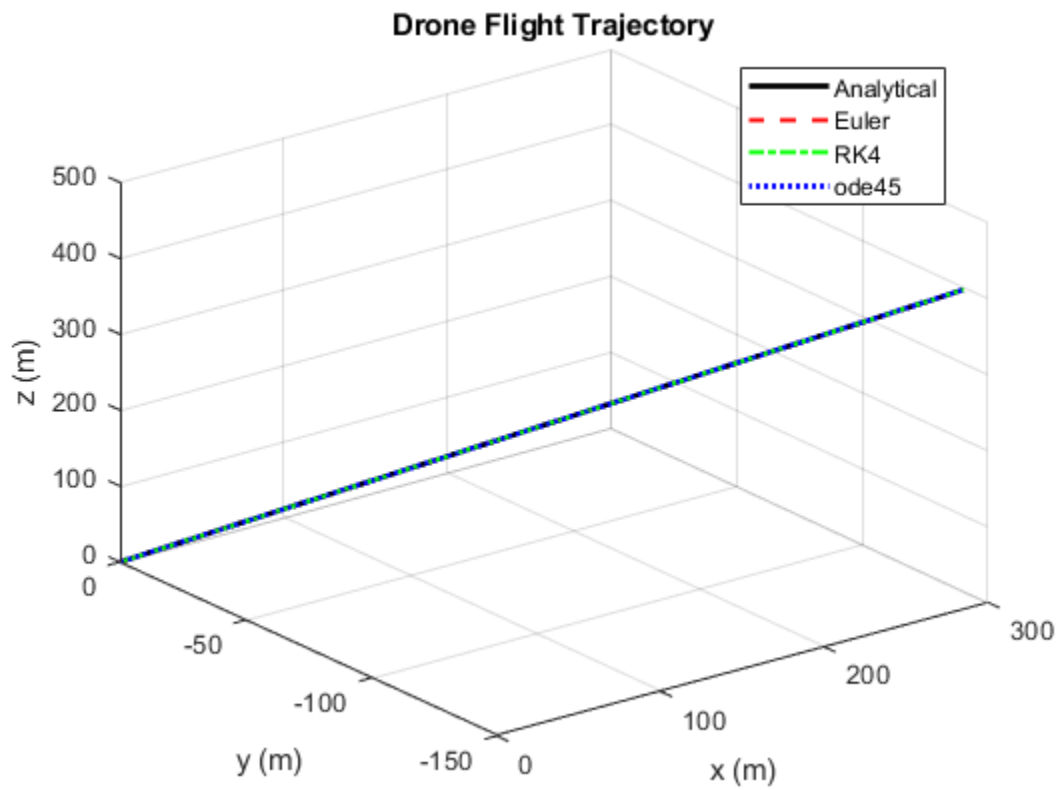
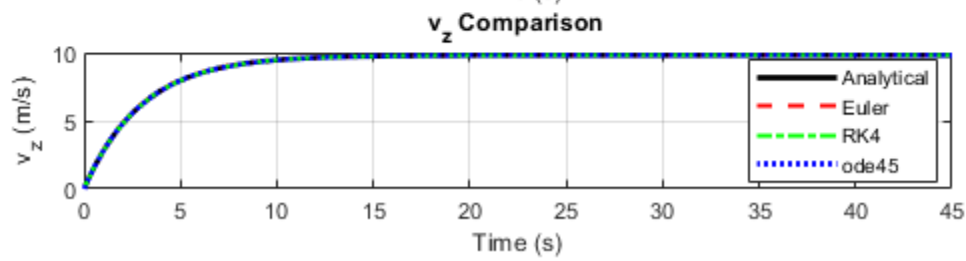
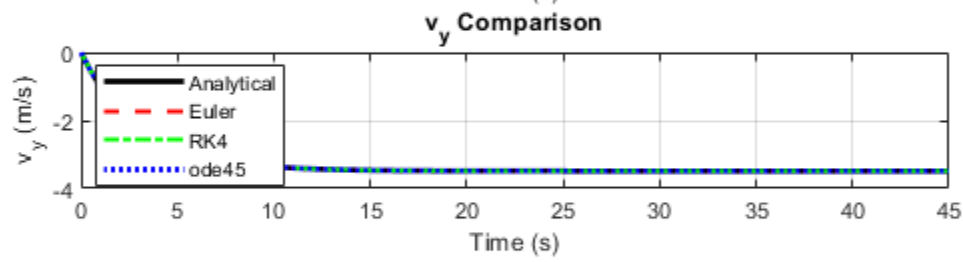
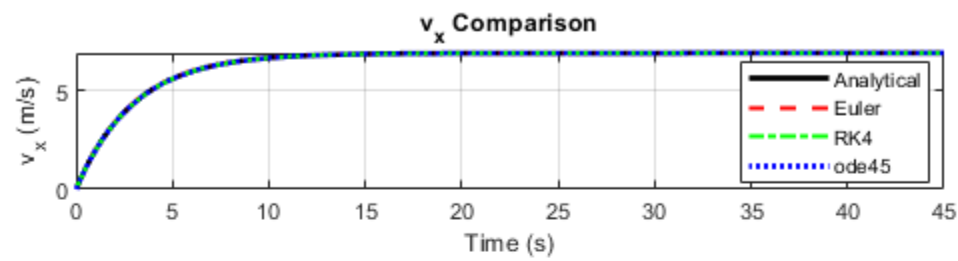
Final State Vector (Euler):

```
disp(YEuler(:,end));
```

4. Visualizations: Trajectory and Velocities

Generates velocity comparisons and 3D trajectory.

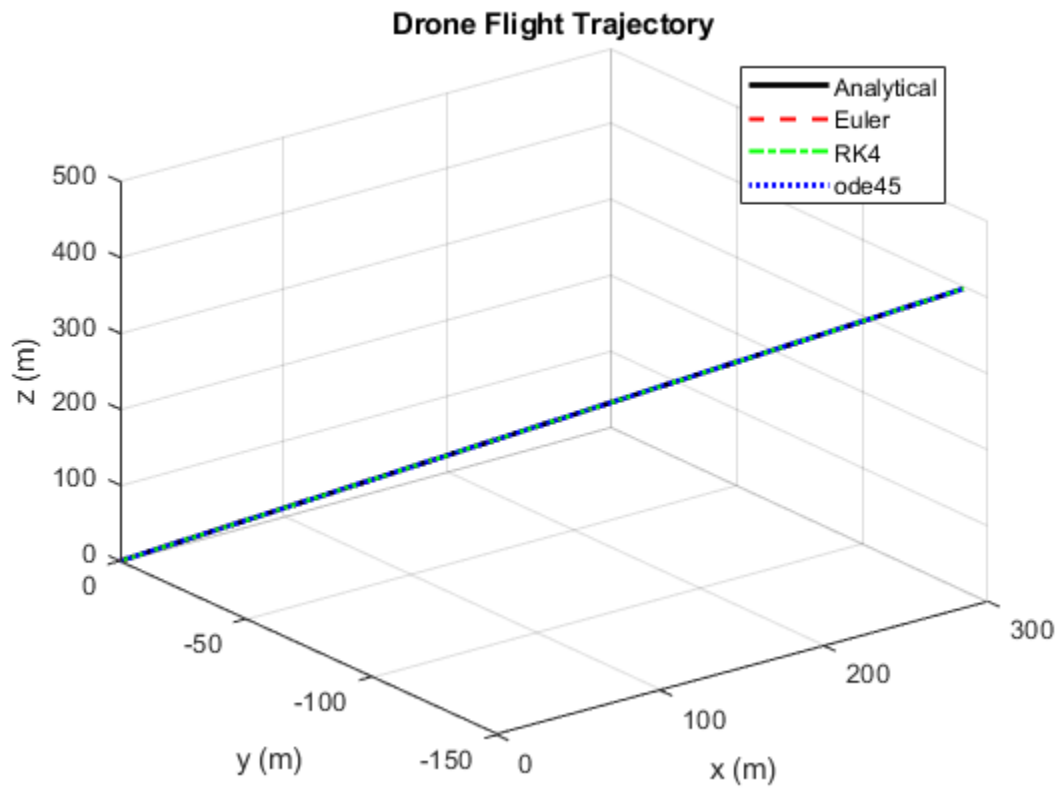
```
plot_results(tAna, YAna, YEuler, YRK4, YODE45);
```

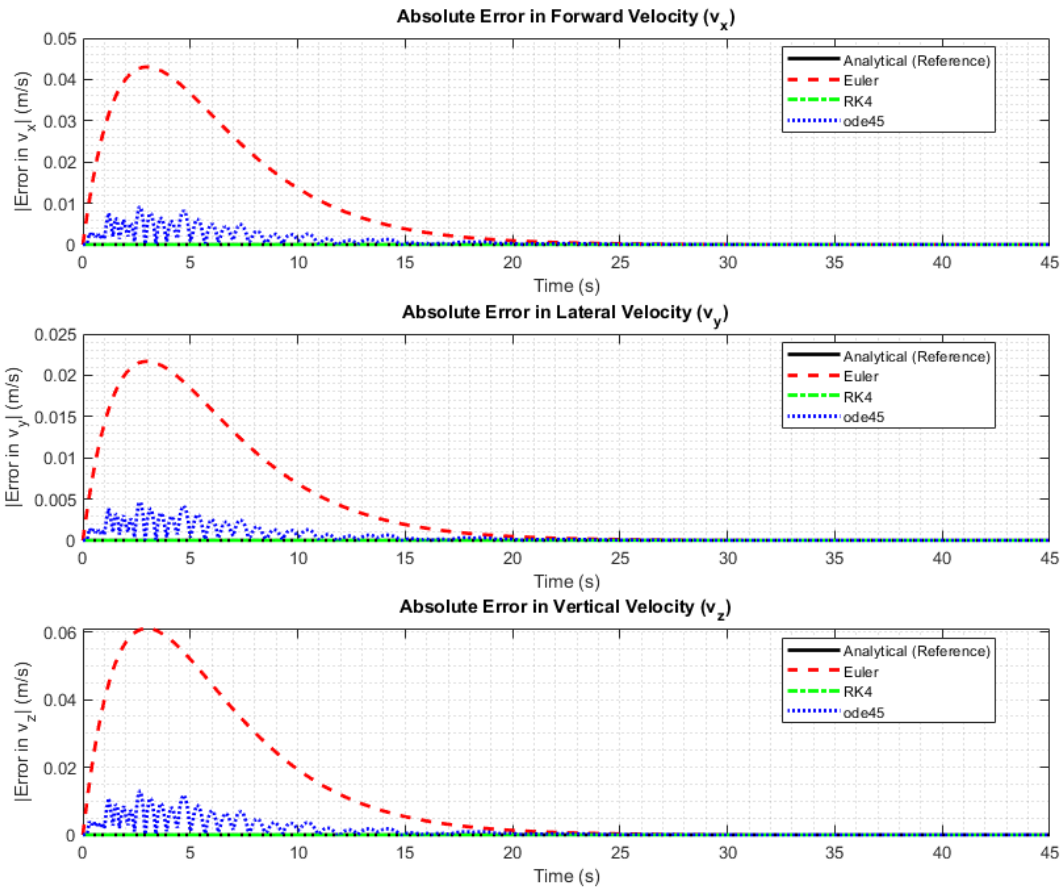


5. Numerical Error Analysis

Compares numerical methods against analytical solution.

```
error_analysis(tAna, YAna, YEuler, YRK4, YODE45);
```

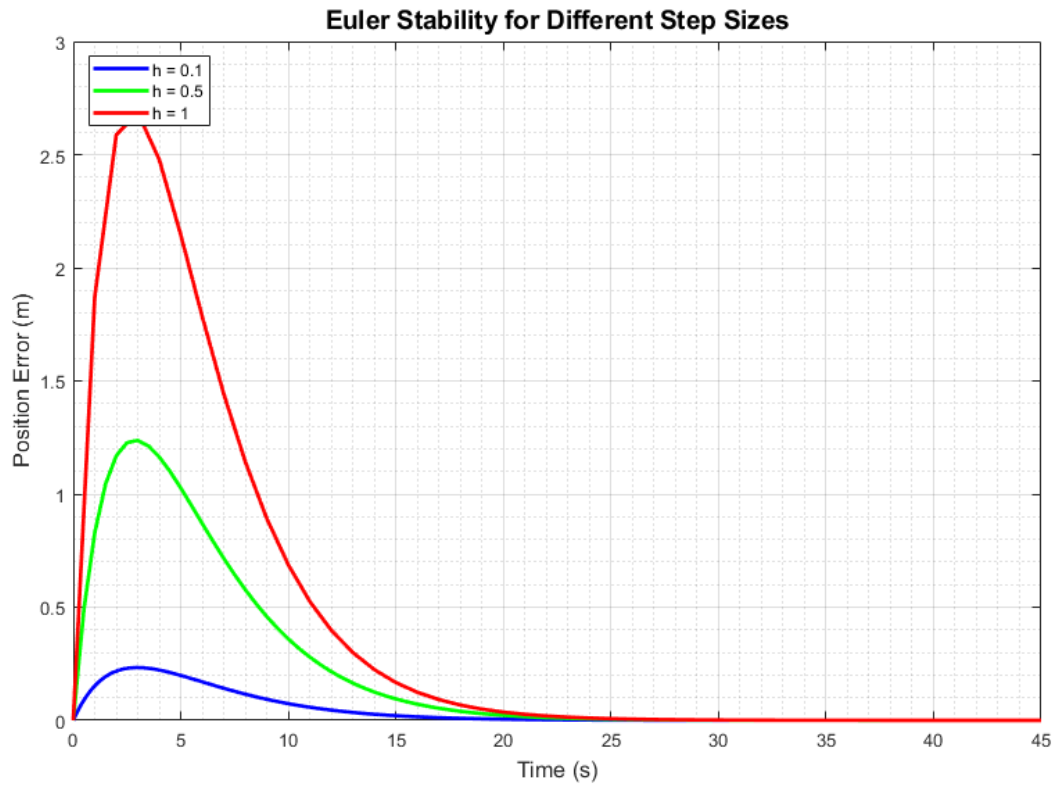




6. Stability Analysis (Euler Step Sizes)

Investigates Euler step sizes.

```
stability_analysis(params);
```



7. Parameter Sensitivity Analysis

Investigates mass, drag, and pitch.

```
sensitivity_analysis(params);
```

PARAMETER STUDY

Mass

1 kg Final Speed : 21.10 m/s

3 kg Final Speed : 21.08 m/s

Drag

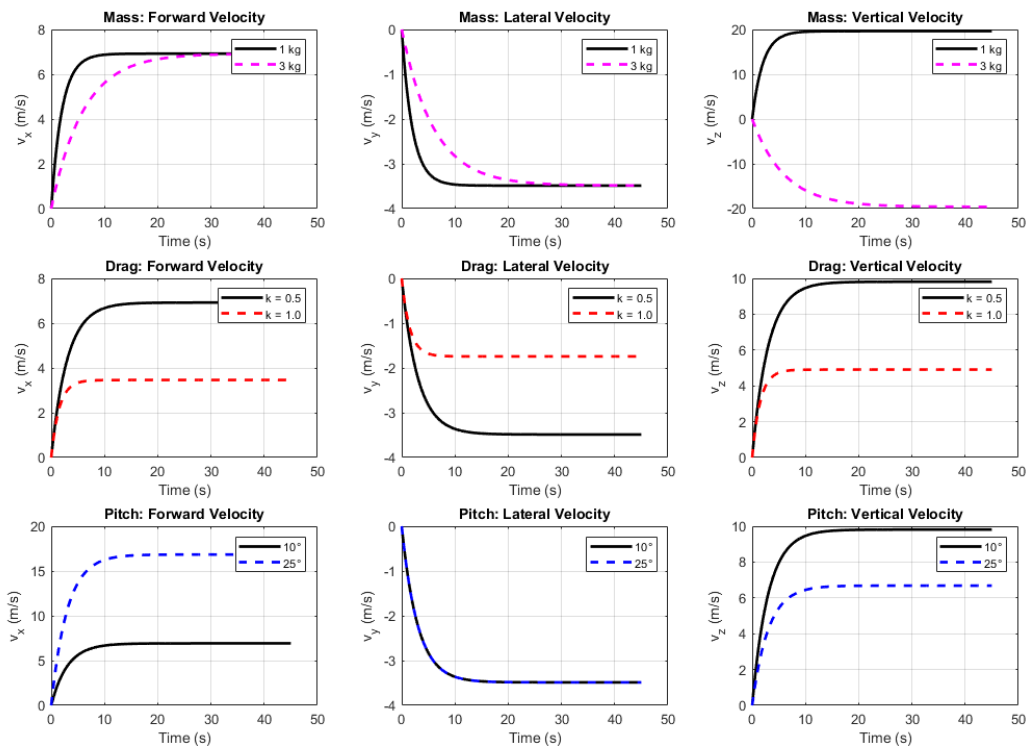
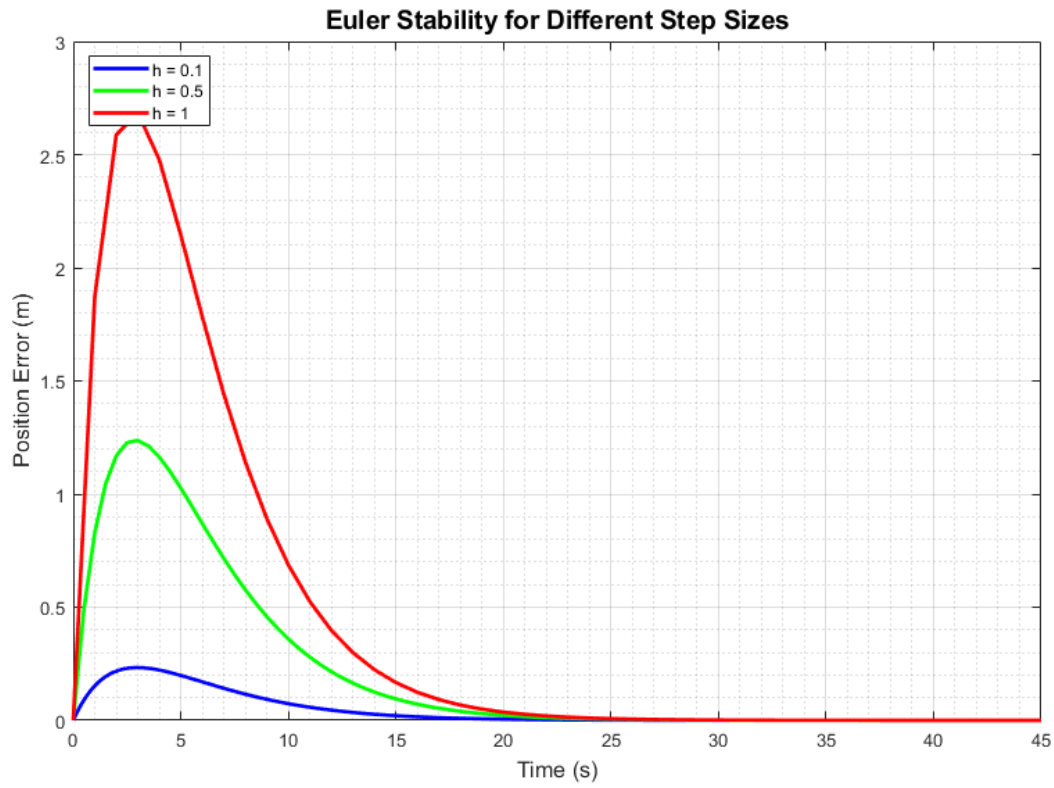
$k = 0.5$ Final Speed : 12.50 m/s

$k = 1.0$ Final Speed : 6.25 m/s

Pitch

10 deg Final Speed : 12.50 m/s

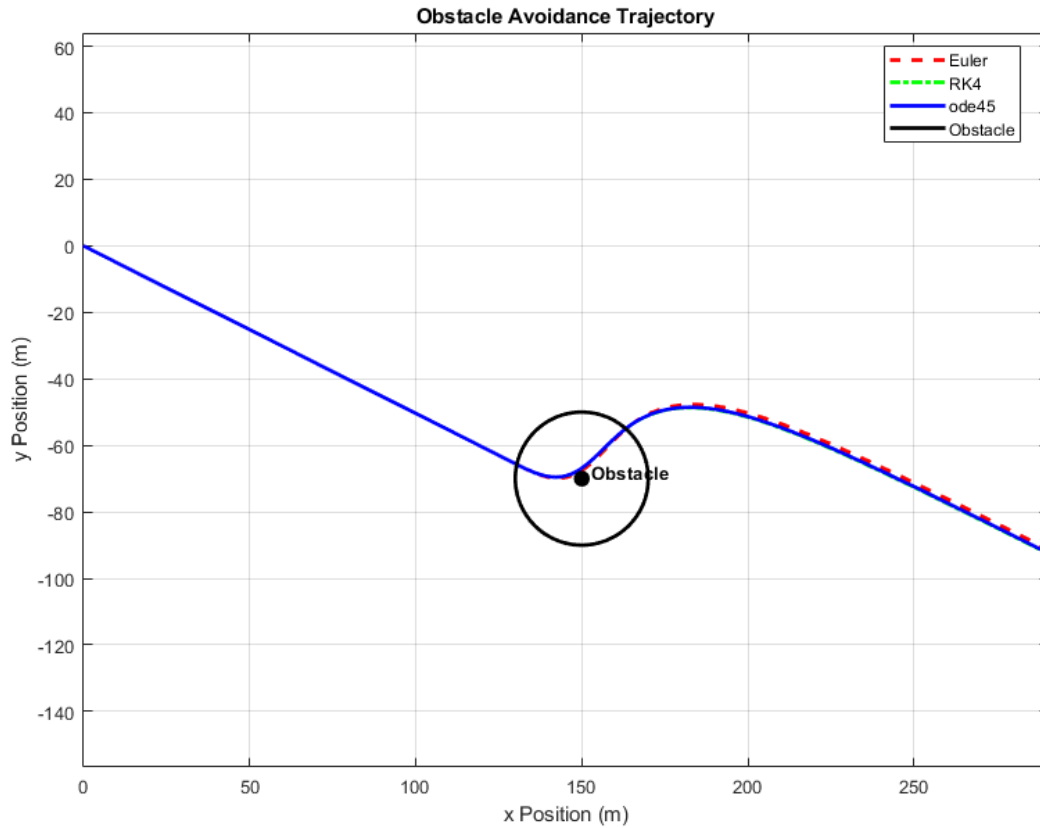
25 deg Final Speed : 18.45 m/s



8. Obstacle Avoidance Simulation

Generates the obstacle avoidance plot.

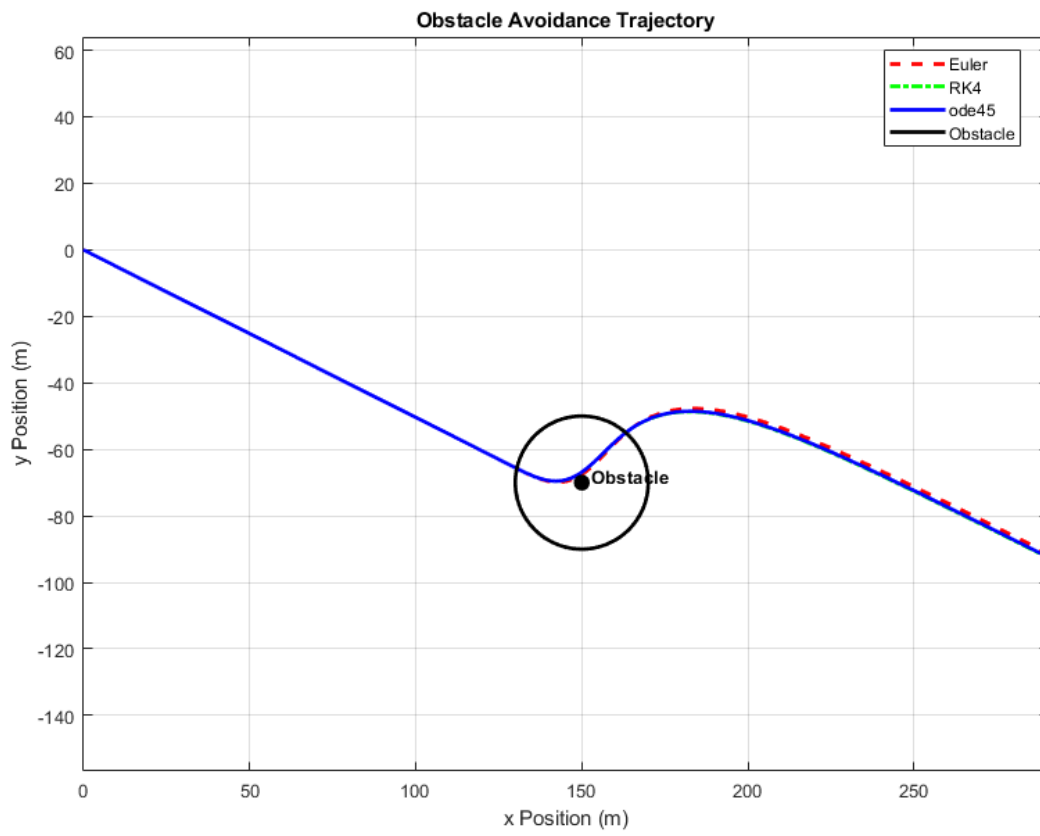
```
obstacle_simulation(params);
```

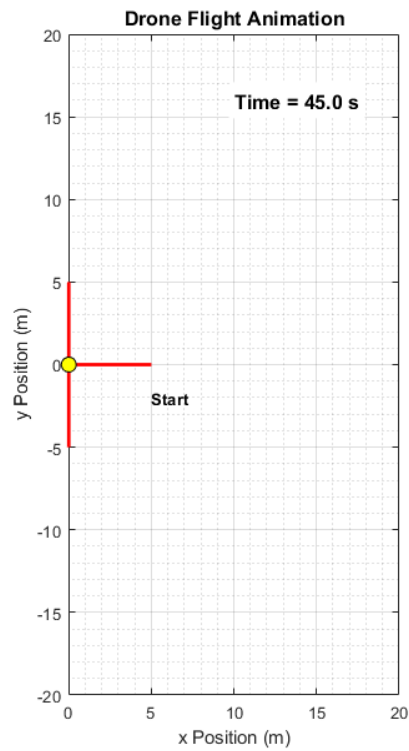


9. Flight Path Animation

Generate the specific trajectory to animate the drone.

```
[~, YObstacle] = euler_solver(@drone_obstacle_dynamics, params);  
animate_drone(YObstacle, params);
```



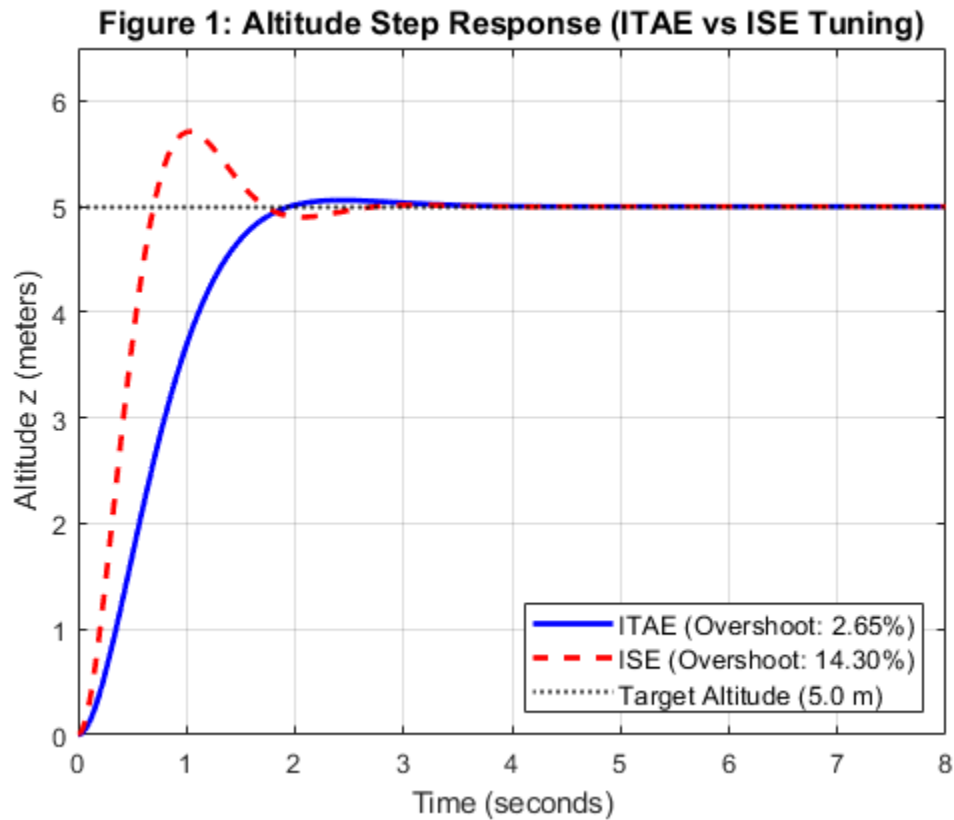


10. Replicate Figure 1: Altitude Step Response

Replicates Figure 11 from the paper (ITAE vs ISE tuning for 5m altitude).

```
replicate_fig1_altitude();
```

Figure 1 (Altitude Step Response) replicated successfully.

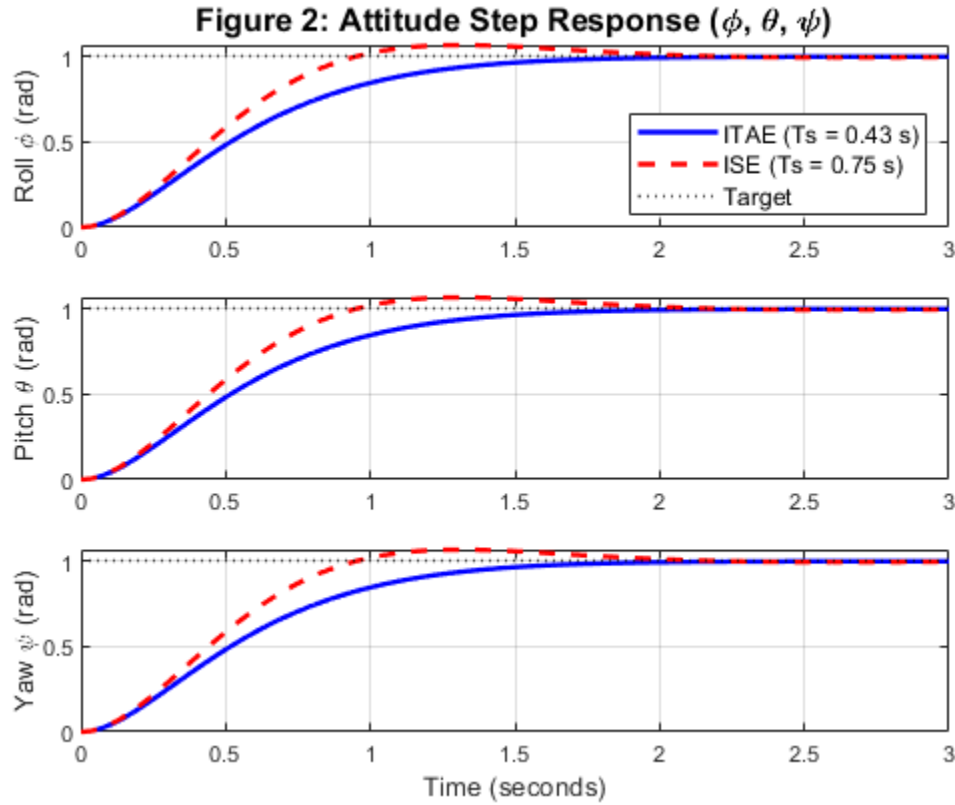


11. Replicate Figure 2: Attitude Tilt Angles

Replicates Figure 12 from the paper (Roll, Pitch, Yaw step responses).

```
replicate_fig2_attitude();
```

Figure 2 (Attitude Step Response) replicated successfully.

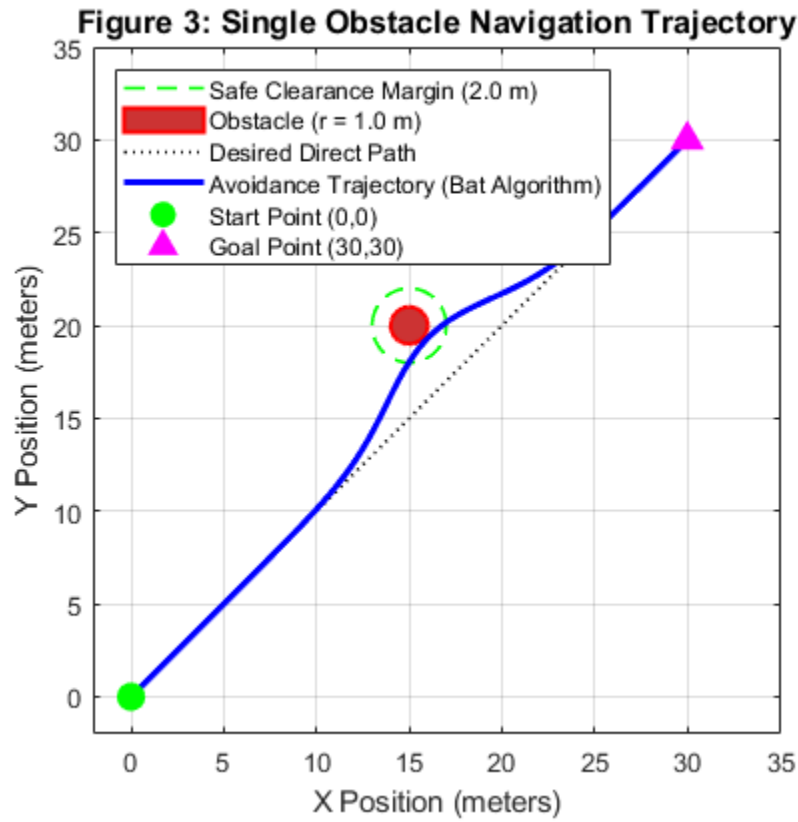
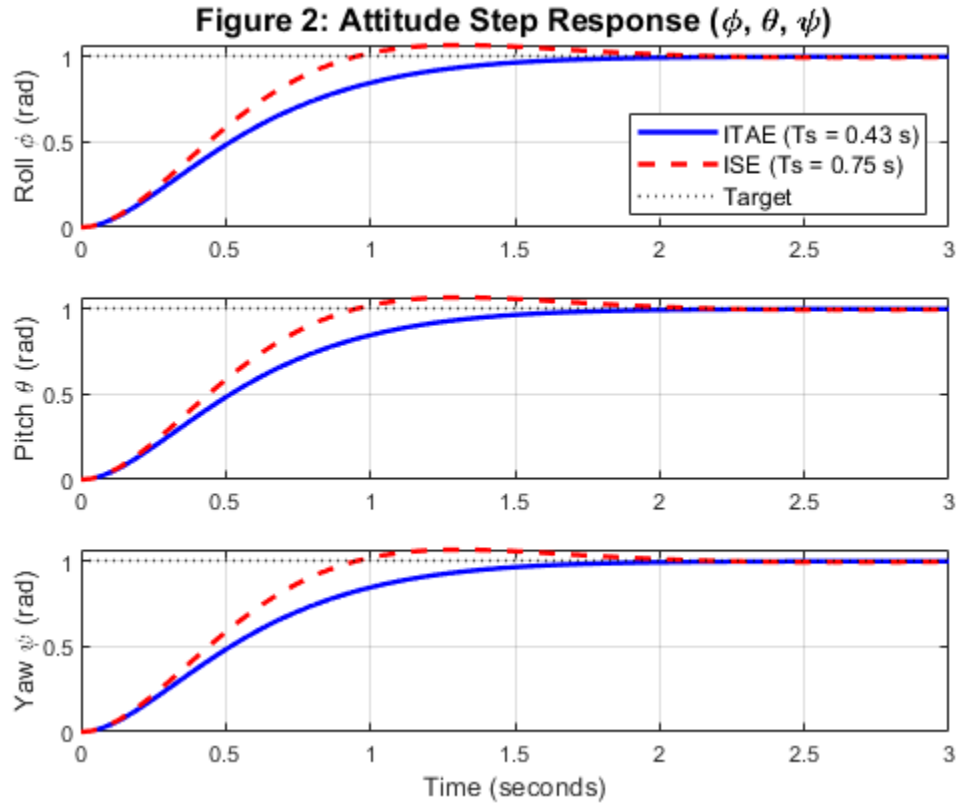


12. Replicate Figure 3: Single Obstacle Avoidance Trajectory

Replicates Figure 13 from the paper (Single obstacle at 15, 20).

```
replicate_fig3_single_obstacle();
```

Figure 3 (Single Obstacle Avoidance) replicated successfully.



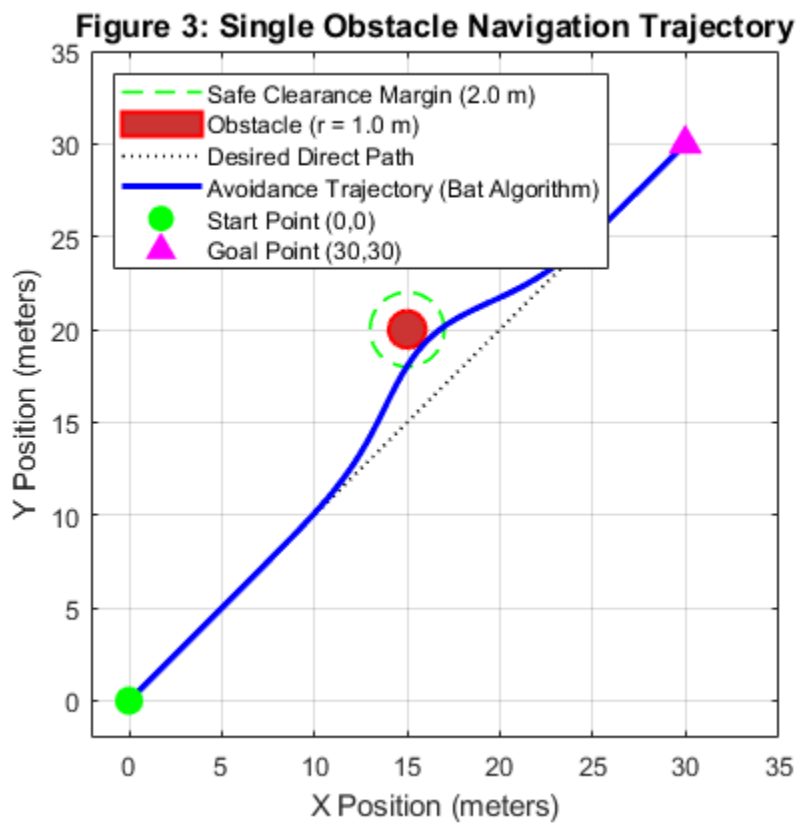
13. Replicate Figure 4: Multiple Obstacle Navigation Trajectory

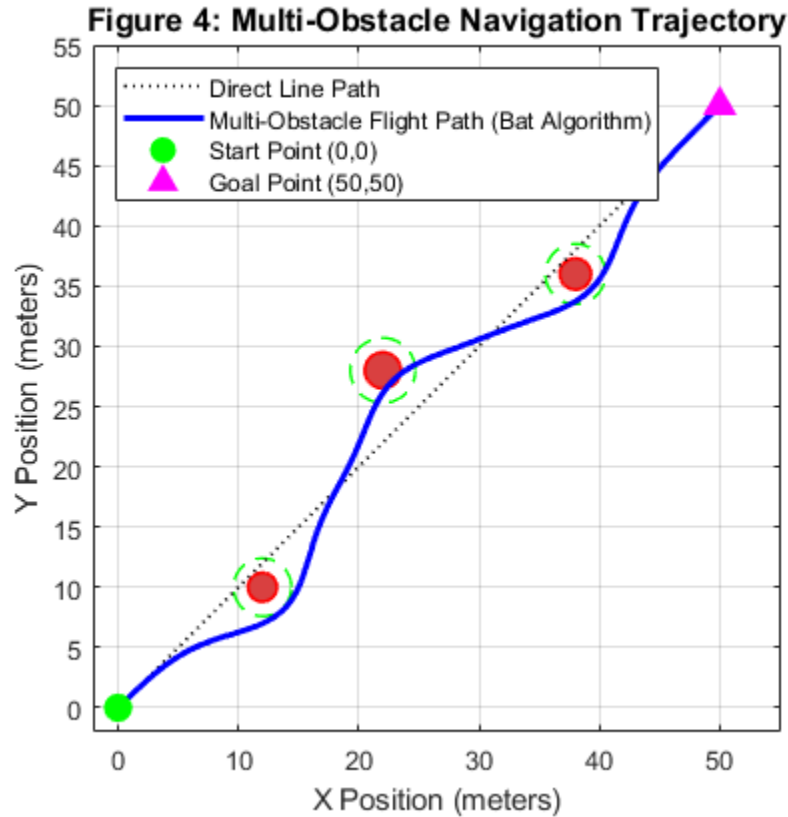
Replicates Figure 14 from the paper (Three consecutive obstacles).

```
replicate_fig4_multi_obstacle();
```

```
disp('All simulations, analyses, and figure replications complete!');
```

*Figure 4 (Multiple Obstacle Navigation) replicated successfully.
All simulations, analyses, and figure replications complete!*





```
=====
```

```
LOCAL FUNCTIONS
```

```
=====
```

```
function dY = drone_trajectory_dynamics(~, Y, params)
% 6-DOF Drone Trajectory Dynamics

m      = params.mass;
T      = params.thrust;
k      = params.drag;
g      = params.gravity;
theta  = params.pitch;
phi    = params.roll;

vx = Y(4);
vy = Y(5);
vz = Y(6);

% Accelerations
ax = (T/m) * sin(theta) * cos(phi) - (k/m) * vx;
ay = -(T/m) * sin(phi) - (k/m) * vy;
az = (T/m) * cos(theta) * cos(phi) - g - (k/m) * vz;
```

```
dY = [vx; vy; vz; ax; ay; az];
end

function dY = drone_obstacle_dynamics(~, Y, params)
% Drone Obstacle Avoidance Dynamics

m = params.mass;
k = params.drag;
g = params.gravity;

vx = Y(4);
vy = Y(5);
vz = Y(6);

% Obstacle position and radius
obs_pos = [150; -70];
R = 20;

dist = norm(Y(1:2) - obs_pos);

if dist < R
    a_avoid = 8 * (1 - dist/R);
else
    a_avoid = 0;
end

ax = - (k/m) * vx;
ay = a_avoid - (k/m) * vy;
az = - g - (k/m) * vz;

dY = [vx; vy; vz; ax; ay; az];
end

function [t, Y] = euler_solver(dynamics, params)
% Euler Solver
t = params.t0:params.h:params.tf;
numSteps = length(t);
Y = zeros(6, numSteps);
Y(:,1) = params.Y0;

for i = 1:numSteps-1
    dY = dynamics(t(i), Y(:,i), params);
    Y(:,i+1) = Y(:,i) + params.h*dY;
end
end

function [t, Y] = rk4_solver(dynamics, params)
% Classical 4th-Order Runge-Kutta Solver
t = params.t0:params.h:params.tf;
numSteps = length(t);
h = params.h;
Y = zeros(6, numSteps);
Y(:,1) = params.Y0;
```

```

for i = 1:numSteps-1
    ti = t(i);
    yi = Y(:,i);

    k1 = dynamics(ti, yi, params);
    k2 = dynamics(ti + 0.5*h, yi + 0.5*h*k1, params);
    k3 = dynamics(ti + 0.5*h, yi + 0.5*h*k2, params);
    k4 = dynamics(ti + h, yi + h*k3, params);

    Y(:,i+1) = yi + (h/6) * (k1 + 2*k2 + 2*k3 + k4);
end
end

function [t, Y] = ode45_solver(dynamics, params)
% MATLAB ode45 Solver
tspan = [params.t0 params.tf];
Y0 = params.Y0;

[t, Y_temp] = ode45(@(t, y) dynamics(t, y, params), tspan, Y0);

% Interpolate to match fixed time grid
t_fixed = params.t0:params.h:params.tf;
Y = interp1(t, Y_temp, t_fixed)';
t = t_fixed;
end

Euler solver completed successfully.
Final State Vector (Euler):
    290.6188
   -146.4217
    412.1212
     6.9195
    -3.4862
     9.8124

```

Published with MATLAB® R2024b